

BoilerBridge: Sprint 1 Retrospective Grading Rubric

Team 26 - BoilerBridge: Xavion Marable, Logan Wilkinson, Emilio Guzman,

Shashank Mukkera

1. What went well? (0.5 point)

- a. You may write this section in sentences and/or list successful user stories and tasks with detailed discussions.
- The backend implementation for the Calendar Event and Must-Haves features worked well. The MongoDB schemas for CalendarEvent and MustHave were successfully designed and integrated with the application, allowing data to be stored and retrieved correctly. The API endpoints for create, read, update and delete operations functioned as expected, and validation checks such as required fields, event time ranges behaved correctly during testing. **(User stories #7 and #8)**
- The authorization logic also worked as intended. Access control checks ensured that only group members could interact with the endpoints and that only creators or group administrators could update or delete items. In addition, the UI was connected to the backend APIs, and users were able to create, edit, or remove events and must-have items through the interface, with changes reflected in the database. **(User stories #7 and #8)**
- The core authentication and user management infrastructure was successfully established during this sprint. The implementation of User Registration **(User Story #1)** and Profile Management **(User Story #2)** provided a functional foundation for the

application, allowing users to create accounts and customize their profiles with images via Cloudinary integration. The implementation of user login (**User Story #13**) contributed to this functional foundation, allowing users to login to their registered accounts and gain access to many more other features. The UI's are also very consistent with the other pages of the website.

- Additionally, the Friend Connections (**User Story #12**) and Student Verification (**User Story #13**) features were integrated effectively. The backend logic for handling .edu email validation and 6-digit verification codes worked as intended, though we haven't set up a proper email service, and the friend request system successfully managed pending, accepted, and blocked states.
- The implementation of the Travel Preferences / Trip Settings feature (**User Story #5**) was completed successfully during this sprint. The `Trip / alltrips` schema was designed and integrated into MongoDB, allowing users to store important travel preferences such as origin, destination, travel dates, budget, transportation options, interests, and group associations. The backend API endpoints for creating and retrieving trip settings were implemented and connected to the UI, enabling users to submit and revisit their saved preferences through a structured form. Validation checks ensured that required fields such as origin, destination, and dates were present and that invalid values, such as negative budgets, were rejected during submission. Authorization logic was also added to ensure that users could only access their own trip settings or trips associated with their group. (**User Story #5**)
- The Activity Reviews feature (**User Story #14**) was also implemented successfully. The Activity schema was extended to support review-related data such as ratings, review counts, and stored review content. A backend endpoint was created to fetch reviews for a given activity, and the UI was updated to display a scrollable list of reviews with

appropriate loading states. The system also handles cases where an activity does not have any available reviews by displaying a user-friendly message instead of an empty interface. Error handling was implemented to ensure that invalid activity IDs or failed data retrieval requests are handled gracefully without breaking the user interface. (**User Story #14**)

- In addition, the Review Summary feature (**User Story #15**) was integrated into the activity interface. Summary fields such as average rating and sentiment highlights were added to the Activity model, and an endpoint was created to generate and retrieve summarized review insights. The UI now displays a review summary card that provides users with quick insights about an activity based on aggregated review data. Caching mechanisms were introduced to avoid repeatedly generating summaries, which improves performance when users revisit the same activity. These features allow users to quickly understand the general sentiment of reviews without needing to read every individual comment. (**User Story #15**)
- The implementation of a forgot password reset (**User Story #4**) was implemented successfully. Password reset fields for a max url lifetime and url generation on the backend from a user submitting the forgot password form worked correctly and consistently. The forgot password and password reset UI pages were easy enough to implement as we already had the login and registration pages to use as templates for these pages. These were also integrated well into the theme of the product as a whole with all of the needed visual features.
- The group messaging page (**User Story #6**) was also integrated into the dashboard and supports messaging between groups. The message schema was created and a list of these objects was added to the TravelGroup schema. Both `/api/goup/[groupId]/messages` POST and GET requests were implemented smoothly into the backend and function as

expected. The UI was not too hard to integrate into the dashboard with most of the dashboard being broken into reusable elements and other pages to use as templates for creating new pages.

- All the functionality regarding travel groups were successfully integrated into the dashboard. The travel groups were added to the database once an authenticated user created them and the users with the right permissions could access them (**User Story #9**). All the features regarding managing travel groups like adding and removing members from the group (User Story #10) were also successfully integrated into the dashboard. All the edge cases regarding transferring group leadership and deleting a group (**User Stories # 11 and #16**) were also implemented and tested.

2. What did not go well? (0.5 point)

- a. Include general retrospective review for this sprint.
 - b. Ensure to list ALL unsuccessful user stories and tasks with detailed discussions. (They should be in line with the Sprint Planning Document for the next sprint.)
- Some parts of the UI implementation became more complicated than necessary, which made debugging and testing more difficult. Although the overall layout and design of the interface looked good, the structure of some components increased complexity and slowed down development when integrating the frontend with the backend APIs. (**User stories #7 and #8**)
 - The duplicate prevention functionality for Must-Have items did not work as expected. The system was intended to prevent users from adding duplicate activities based on the same placeID or the same name and address, but during testing, this validation did not consistently block duplicates. This means that the acceptance criterion related to

duplicate prevention for Must-Haves was not fully satisfied. (**User stories #7 and #8**)

- While the backend logic for profile updates was solid, the integration of Cloudinary for image uploads (**User Story #2**) presented more challenges than I thought. I wanted live updates for everything, and wanted it to load quickly, so making sure that the navbar and profile page synced instantly without flickering on the page required a lot of refactors of the Dashboard and layout.tsx files.
- The Friend Search (**User Story #12**) UI also faced hurdles with making it real-time. Specifically, providing immediate visual confirmation when a request was sent versus when a user was already a friend needed extra state management that slowed down the frontend development.
- While the backend implementation for the Activity Reviews and Review Summary features (**User Stories #14 and #15**) worked correctly, the frontend styling and UI presentation could be improved. The current layout successfully displays reviews and summary information, but the visual structure of the components could be refined to better match the rest of the interface and improve readability. Some elements such as spacing, review card formatting, and overall responsiveness could be improved to create a more polished user experience. (**User Stories #14 and #15**)
- Another area that could be improved is testing coverage and robustness. While the core functionality was implemented and basic validation was performed, the testing process could have included more edge cases and integration tests to ensure stability across different scenarios. For example, additional tests for large review datasets, API failures, and concurrent user access would help ensure that the system behaves reliably under a wider range of conditions. (**User Stories #14 and #15**)
- Additionally, for the Activity Reviews feature (**User Story #14**), the current implementation relies on a limited set of activity data. In the next sprint, I plan to

incorporate an additional external activities API to provide users with a broader and more diverse set of activity options. This would improve the usefulness of the reviews feature by allowing users to explore a wider range of activities and see reviews for more locations. (**User Story #14**)

- While the implementation of login (**User Story #3**) was implemented correctly and all acceptance criteria were met. The other part of the user story, implementing user settings, was not able to be completed during this first sprint. I plan to spend extra time next sprint to complete these user stories and work more to hit all of the acceptance criteria for my assigned user stories. This would complete my unfinished user stories from the first sprint and allow me to continue on and complete my newly assigned user stories.
- Additionally, the implementation of real time messaging was not implemented(**User Story #15**). The rest of the acceptance criteria was met except the integration of real time messaging and required a page reload to load newly sent messages. Next sprint I need to set aside a larger amount of time to decide how we want to accomplish this task based on tradeoffs of certain methods as this last sprint most of my research about the topic was in small sprints of time and I never got a solid understanding of the options.

3. How should you improve? (1.0 point)

- a. Mention at least two ways to improve your work in the next sprint and be as detailed as you can.
- One improvement for the next sprint would be to simplify the UI implementation by designing frontend components more carefully and breaking them into smaller, modular pieces. This would reduce unnecessary complexity and make it easier to debug and maintain the code. We will also move all Zod schemas into a single shared directory instead of keeping them inside individual components. This makes sure that the exact

same validation logic is applied to both the client-side forms and the server-side API routes, preventing inconsistencies and making the code much easier to reuse for new features.

- Another improvement would be to perform integration and validation testing earlier in the development process. Testing features such as duplicate prevention and API validation earlier would help identify issues sooner and ensure that acceptance criteria are fully satisfied before the sprint concludes.

4. Notes

- a. For section 1 and 2, only listing out some user stories and tasks without any discussion will not get you full credits.
- b. Please make sure to include your project name, team number and the names of all your team members.

Total: 2 points (2% of your project grade)